

CAS4160 Homework 4: Soft Actor-Critic

[Your name]
[Your student ID]

1 Introduction

DQN은 이산 행동 공간(discrete action spaces)에 매우 적합합니다. 그러나 $\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$ 를 계산해야 하는 문제가 있습니다. 이산 행동 공간에서는 이를 쉽게 수행할 수 있는데, n 개의 행동 중에서 Q-value가 가장 높은 것을 모두 탐색(exhaustively)하여 찾을 수 있기 때문입니다. 반면, 연속 행동 공간(continuous action spaces)에서는 이는 복잡한 비선형 최적화 문제가 될 수 있습니다.

Actor-critic 방법들은 명시적인 정책(explicit policy) π 를 학습함으로써 이 문제를 우회합니다. 이 정책은 $\mathbb{E}_{\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})} Q(\mathbf{s}, \mathbf{a})$ 를 최대화하도록 훈련됩니다. 이 과제를 통해, 여러분은 이 계열(line)에서 가장 인기 있는 알고리즘 중 하나인 Soft Actor-Critic (SAC)을 구현하는 방법을 배우게 될 것입니다.

이 과제의 모든 실험은 다음 명령어를 통해 실행할 수 있습니다:

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/<CONFIG>.yaml
```

2 Code Structure Overview

코드 구조는 이전 과제와 유사합니다. 이 과제에서는 주로 다음 코드들과 작업하게 됩니다:

- `cas4160/scripts/run_hw4.py`: SAC 구현을 위한 주요 훈련 루프.
- `cas4160/agents/sac_agent.py`: 여러분이 구현할 SAC 학습자.
- `cas4160/networks/state_action_value_critic.py`: 간단한 MLP 기반 critic 네트워크, $Q(\mathbf{s}, \mathbf{a})$. DQN의 critic은 상태를 모든 행동에 대한 Q-value 배열로 매핑하는 반면, 이 critic은 $(\mathbf{s}, \mathbf{a})$ 를 단일 Q-value로 매핑한다는 점에 유의하세요.
- `cas4160/env_configs/sac_config.py`: 기본 구성 (및 하이퍼파라미터 목록).
- `experiments/sac/*.yaml`: 실험을 위한 구성 파일.

3 Training Loop

SAC를 구현하기 전에, 먼저 `cas4160/scripts/run_hw4.py`의 모든 TODO를 완료해야 합니다. 이는 Homework 3의 DQN 스크립트와 매우 유사합니다. 두 알고리즘 모두 off-policy 방법이기 때문입니다!

4 Bootstrapping

DQN에서와 마찬가지로, 우리는 타겟 critic으로부터 “부트스트래핑”하여 critic을 훈련합니다. 튜플 $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}, d_t)$ (여기서 d_t 는 해당 전이 후 트라젝트리가 종료되는지를 나타내는 플래그입니다)를 사용하여, 타겟 값 y 를 다음과 같이 설정합니다:

$$y \leftarrow r_t + \gamma(1 - d_t)Q_{\phi}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}), \text{ where } \mathbf{a}_{t+1} \sim \pi(\mathbf{a}_{t+1} | \mathbf{s}_{t+1}).$$

그런 다음, 이 부트스트래핑된 타겟 y 를 사용하여 L2 손실을 최소화함으로써 우리의 critic Q_ϕ 를 업데이트합니다:

$$\min_{\phi} (Q_{\phi}(\mathbf{s}_t, \mathbf{a}_t) - y)^2.$$

실제로는 학습을 안정화하기 위해 별도의 타겟 네트워크, $Q_{\phi'}$ 를 사용합니다. 강의에서 다룬 바와 같이, 타겟 네트워크를 업데이트하는 두 가지 일반적인 전략이 있습니다:

- **하드 업데이트** (DQN에서 구현됨): 매 K 스텝마다 $\phi' \leftarrow \phi$ 로 설정합니다.
- **소프트 업데이트**: ϕ' 는 *Polyak averaging* (지수 이동 평균)으로 ϕ 쪽으로 지속적으로 업데이트됩니다. 각 업데이트 후, 다음 연산을 수행합니다 (τ 는 일반적으로 매우 작은 값, 예: 0.005):

$$\phi' \leftarrow \phi' + \tau(\phi - \phi').$$

4.1 Implementation

먼저 `cas4160/agents/sac_agent.py`에서 critic 업데이트를 구현해 보겠습니다:

- 위에서 설명한 부트스트래핑된 critic 업데이트를 `update_critic` 메서드에 구현하세요. 이 시점에서는 `next_action_entropy`를 구현할 필요가 없습니다.
- `update` 메서드에서 critic을 `num_critic_updates` 번 업데이트하세요.
- `update`에서 소프트와 하드 타겟 네트워크 업데이트를 구현하세요.

4.2 Experiments

- `experiments/sac/sanity_pendulum_1.yaml` 설정을 사용하여 Pendulum-v1 환경에서 에이전트를 훈련하세요.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/sanity_pendulum_1.yaml
```

- 아직 actor를 훈련하지 않았기 때문에 높은 보상을 얻지 못할 것입니다. 하지만 Q-value는 상당히 큰 음수(예: -700)로 안정화되어야 합니다. Q-value가 마이너스 무한대로 가거나 0 근처에 머무른다면 코드에 문제가 있을 수 있습니다.
- 이 환경에서 do-nothing" 보상은 스텝당 약 -10입니다. 에이전트가 항상 do nothing"을 한다고 가정하면, 에이전트는 매 스텝 -10을 받고 절대 종료되지 않습니다. discount factor $\gamma = 0.99$ 를 고려할 때, 훈련 배치에서 기대되는 평균 Q-value는 대략 얼마가 되어야 하는지 계산하세요.

답변: **설명을 추가하세요.**

5 Entropy Bonus and Soft Actor-Critic

DQN에서는 주어진 시간에 어떤 행동을 취할지 결정하기 위해 ϵ -greedy 전략을 사용합니다. 연속 공간에서는 탐험 노이즈를 생성하는 여러 옵션이 있습니다.

가장 일반적인 접근법 중 하나는 actor가 높은 엔트로피(즉, 더 무작위하게 되도록)하도록 장려하는 *entropy bonus*를 제공하는 것입니다. 이는 "temperature" 계수 β 로 스케일링됩니다. 예를 들어, REPARAMETRIZE 경우:

$$\mathcal{L}_{\pi} = Q(\mathbf{s}, \mu_{\theta}(\mathbf{s}) + \sigma_{\theta}(\mathbf{s})\epsilon) + \beta \mathcal{H}(\pi_{\theta}(\cdot | \mathbf{s})), \text{ where } \epsilon \sim \mathcal{N}(0, 1), \quad (1)$$

여기서 엔트로피는 $\mathcal{H}(\pi(\cdot | \mathbf{s})) = \mathbb{E}_{\mathbf{a} \sim \pi} [-\log \pi(\mathbf{a} | \mathbf{s})]$ 로 정의됩니다.

엔트로피 항이 Q-function에도 포함되도록 하기 위해, soft Q target 값을 사용합니다:

$$y \leftarrow r_t + \gamma(1 - d_t) [Q_\phi(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) + \beta \mathcal{H}(\pi(\cdot | \mathbf{s}_{t+1}))]. \quad (2)$$

이 soft Q-value들은 actor가 어떤 행동을 취해야 할지 확신이 없을 때 더 무작위한 행동을 선택하는 행동을 초래합니다. 더 자세한 내용은 SAC 논문: <https://arxiv.org/abs/1801.01290>을 참조하세요.

엔트로피 $\mathcal{H}(\pi_\theta) = -\mathbb{E}[\log \pi_\theta]$ 를 최대화하려면 샘플링 분포를 통해 미분해야 합니다. 이는 강의에서 다룬 “reparametrization trick”을 통해 할 수 있습니다 – 새로그침이 필요하다면 Section 7를 참조하세요.

5.1 Implementation

이 섹션에서는 정책의 엔트로피를 계산(추정)하고 `cas4160/agents/sac_agent.py`에서 soft Q target 값을 구현해야 합니다:

- 하나의 샘플을 사용하여 행동 분포의 엔트로피를 근사적으로 계산하도록 `entropy` 메서드를 구현하세요:

$$\mathcal{H}(\pi_\theta(\cdot | \mathbf{s})) = \mathbb{E}_{\mathbf{a} \sim \pi_\theta} [-\log \pi_\theta(\mathbf{a} | \mathbf{s})] \approx -\log \pi_\theta(\hat{\mathbf{a}} | \mathbf{s}), \text{ where } \hat{\mathbf{a}} \sim \pi_\theta(\cdot | \mathbf{s}). \quad (3)$$

- `update_critic`에서 타겟 critic 값에 엔트로피 항을 추가하세요.
- `update`에서 `update_actor`를 호출하여 actor가 엔트로피를 최대화하도록 훈련하세요.

5.2 Experiments

- `experiments/sac/sanity_pendulum_2.yaml` 설정을 사용하여 Pendulum-v1 환경에서 soft Q-value를 학습하고 정책 엔트로피를 최대화하는 에이전트를 훈련하세요:

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/sanity_pendulum_2.yaml
```

- 실험은 actor 업데이트 동안 `entropy`를 로그로 기록합니다. 에이전트가 엔트로피만 최대화하므로, 1차원 행동 공간에서 가능한 최대 엔트로피(에 가까운 값)를 달성해야 합니다. 엔트로피는 uniform distribution 일 때 최대화됩니다:

$$\mathcal{H}(\mathcal{U}[-1, 1]) = \mathbb{E}[-\log p(x)] = -\log \frac{1}{2} = \log 2 \approx 0.69.$$

현재 actor loss가 오직 entropy bonus로만 구성되어 있기 때문에 (아직 보상을 최대화하는 어떤 것도 구현하지 않았습니다), 엔트로피는 대략 이 수준에 도달할 때까지 증가해야 합니다. 엔트로피가 이 값보다 높거나 상당히 낮다면 코드에 문제가 있을 수 있습니다.

- 이 섹션에서는 보고할 결과가 없습니다.

6 Actor Update with REINFORCE

보상을 최대화하도록 actor를 훈련하기 위해, Homework 2에서 사용했던 REINFORCE gradient estimator를 사용할 수 있습니다!

정책의 목적은 기대 수익(expected return)을 최대화하는 것이며, 우리는 그 gradient를 필요로 합니다:

$$\nabla_\theta \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi_\theta(\mathbf{a} | \mathbf{s})} [Q_\phi(\mathbf{s}, \mathbf{a})].$$

강의와 Homework 2에서 배운 바와 같이, REINFORCE를 사용하여 policy gradient를 계산할 수 있습니다:

$$\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi_\theta(\mathbf{a} | \mathbf{s})} [\nabla_\theta \log(\pi_\theta(\mathbf{a} | \mathbf{s})) Q_\phi(\mathbf{s}, \mathbf{a})].$$

a 행동들은 π_θ 로부터 샘플링되며, 실제 데이터 샘플은 **필요하지 않습니다**. 이는 이 policy gradient의 분산을 줄이기 위해, 주어진 어떤 상태에 대해서도 π_θ 로부터 더 많은 행동을 샘플링할 수 있다는 것을 의미합니다! 여러분은 코드에서 `num_actor_samples` 파라미터를 사용하여 이를 구현하게 될 것입니다.

6.1 Implementation

이 섹션에서는 `cas4160/agents/sac_agent.py`에서 REINFORCE policy gradient estimator를 구현해야 합니다:

- `actor_loss_reinforce` 메서드에서 REINFORCE gradient estimator를 구현하세요.

6.2 Experiments

- 다음 실험 각각은 **최소 한 시간 이상** 소요된다는 점에 유의하세요.
- `sanity_invertedpendulum_reinforce.yaml`을 사용하여 `InvertedPendulum-v4` 환경에서 에이전트를 훈련하세요. **모든 타임스텝 동안 똑바로 서 있는 상태를 유지하는 것을 의미하는 총 보상 1000에 가까운 값을 달성**해야 합니다. 다만, `return`은 변동될 수 있으며, 가끔 1000 아래로 떨어지는 것은 허용됩니다.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/
sanity_invertedpendulum_reinforce.yaml
```

답변:

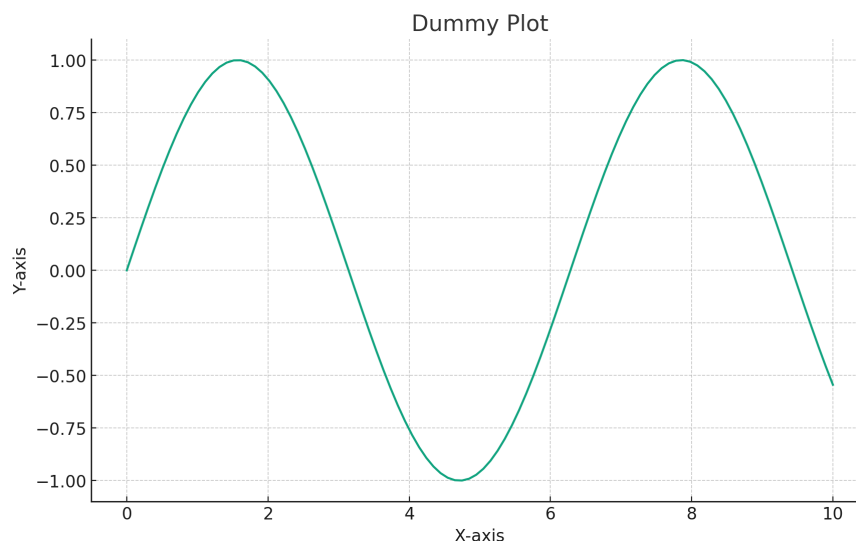


Figure 1: 이 플롯을 여러분의 플롯으로 교체하고, 캡션을 추가하세요.

설명을 추가하세요.

- 제공된 설정(`halfcheetah_reinforce1.yaml`)을 사용하여 `HalfCheetah-v4` 환경에서 에이전트를 훈련하세요. 이 설정은 policy gradient 추정을 위해 단 하나의 샘플링된 행동만 사용한다는 점에 유의하세요. **결과는 500K 스텝 내에 양의 보상에 도달해야 합니다**.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/halfcheetah_reinforce1.
yaml
```

- halfcheetah_reinforce10.yaml을 사용하여 또 다른 에이전트를 훈련하세요. 이 설정은 REINFORCE gradient를 계산하기 위해 actor로부터 10개의 행동 샘플을 취합니다. 이를 REINFORCE-10이라고 부르고, 위의 단일 샘플 버전을 REINFORCE-1이라고 부릅니다. **REINFORCE-10은 200K 스텝 내에 500 이상의 evaluation return을 달성해야 합니다.**
- REINFORCE-1과 REINFORCE-10의 결과(시간에 따른 evaluation return)를 플롯으로 그리세요. 결과를 비교하고 설명하세요.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/halfcheetah_reinforce10.yaml
```

답변:

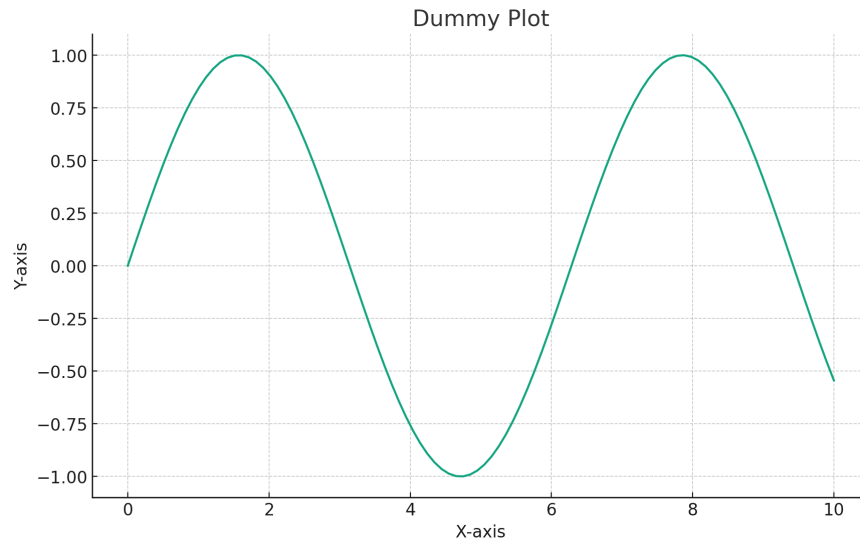


Figure 2: 이 플롯을 여러분의 플롯으로 교체하고, 캡션을 추가하세요.

설명을 추가하세요.

7 Actor Update with REPARAMETRIZE

REINFORCE는 많은 샘플과 함께 꽤 잘 작동하지만, 특히 고차원 행동 공간에서는 낮은 분산을 주기 위해 상당히 많은 샘플을 필요로 하기 시작합니다. 우리는 재매개변수화된 gradient를 사용하여 이를 개선할 수 있습니다.

π_θ 를 $\mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s})\epsilon$ 로 매개변수화하면 (여기서 ϵ 은 정규 분포를 따릅니다), 다음과 같이 쓸 수 있습니다:

$$\nabla_{\theta} \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] = \nabla_{\theta} \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [Q(\mathbf{s}, \mu_{\theta}(\mathbf{s}) + \sigma_{\theta}(\mathbf{s})\epsilon)] = \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [\nabla_{\theta} Q(\mathbf{s}, \mu_{\theta}(\mathbf{s}) + \sigma_{\theta}(\mathbf{s})\epsilon)] \quad (4)$$

이 gradient estimator는 종종 훨씬 낮은 분산을 제공하므로, 적은 수의 샘플로도 사용할 수 있습니다 (실제로는 단일 샘플만 사용해도 매우 잘 작동합니다).

7.1 Implementation

- cas4160/agents/sac_agent.py에서 actor_loss_reparametrize()를 구현하세요. 샘플링에 reparametrization trick을 사용하도록 주의하세요 (*rsample())!

7.2 Experiments

- 다음 실험 각각은 최소 한 시간 이상 소요된다는 점에 유의하세요.
- `sanity_invertedpendulum_reparametrize.yaml`을 사용하여 `InvertedPendulum-v4`를 해결할 수 있는지 확인하세요. 모든 타임스텝 동안 똑바로 서 있는 상태를 의미하는 보상 1000에 가까운 값을 달성해야 합니다.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/  
sanity_invertedpendulum_reparametrize.yaml
```

- `halfcheetah_reparametrize.yaml`을 사용하여 `HalfCheetah-v4`에서 (다시 한 번) 훈련하세요. 세 가지 **gradient estimator** 모두 (REINFORCE-1, REINFORCE-10, 그리고 REPARAMETRIZE)의 결과를 동일한 축 세트에 플롯으로 그리세요. x -축은 environment steps 수, y -축은 evaluation return입니다.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/halfcheetah_reparametrize.  
yaml
```

답변:

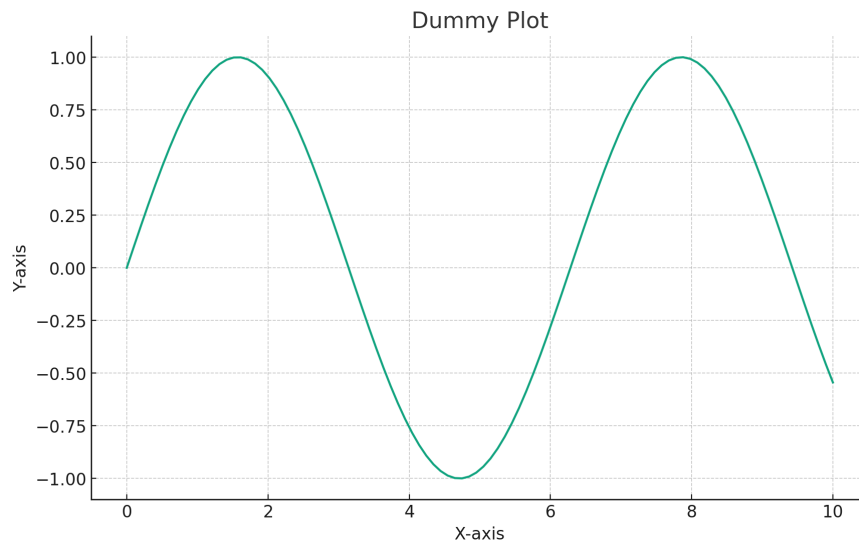


Figure 3: 이 플롯을 여러분의 플롯으로 교체하고, 캡션을 추가하세요.

설명을 추가하세요.

- (선택 사항) `humanoid.yaml`을 사용하여 `Humanoid-v4` 환경에서 에이전트를 훈련하고 결과를 플롯으로 그리세요.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/humanoid.yaml
```

8 Stabilizing Target Values

DQN에서와 마찬가지로, 단일 critic을 사용한 타겟 Q 는 *overestimation bias* (과대평가 편향)를 보입니다! 이를 해결하기 위해 일반적으로 사용되는 몇 가지 전략이 있습니다:

- **Double-Q**: 두 개의 critic Q_{ϕ_A}, Q_{ϕ_B} 를 학습하고, 두 개의 타겟 네트워크 $Q_{\phi'_A}, Q_{\phi'_B}$ 를 유지합니다. 그런 다음, Q_{ϕ_B} 에 대한 타겟 값을 계산할 때는 $Q_{\phi'_A}$ 를 사용하고, 반대로 Q_{ϕ_A} 에 대한 타겟 값을 계산할 때는 $Q_{\phi'_B}$ 를 사용합니다 (이는 Double DQN과 약간 다릅니다): $y_A = r + \gamma Q_{\phi'_B}(s', a')$ $y_B = r + \gamma Q_{\phi'_A}(s', a')$
- **Clipped double-Q**: 두 개의 critic Q_{ϕ_A}, Q_{ϕ_B} 를 학습합니다 (그리고 두 개의 타겟 네트워크를 유지합니다). 그런 다음, 타겟 값을 $\min(Q_{\phi'_A}, Q_{\phi'_B})$ 로 계산합니다. $y_A = y_B = r + \gamma \min(Q_{\phi'_A}(s', a'), Q_{\phi'_B}(s', a'))$

8.1 Implementation

- `sac_agent.py`의 `q_backup_strategy` 함수에서 double-Q와 clipped double-Q를 구현하세요.

8.2 Experiments

- 다음 실험 각각은 약 20분 정도 소요된다는 점에 유의하세요.
- single-Q, double-Q, clipped double-Q를 해당 설정 파일을 사용하여 Hopper-v4 환경에서 실행하세요. 어느 것이 가장 잘 작동하나요? 각각의 `eval_return`과 `q_values`를 로그로 기록한 값을 플롯으로 그리세요. 이러한 결과가 overestimation bias와 어떻게 관련되는지 논의하세요. 기본 seed가 잘 작동할 것이지만, 그렇지 않다면 더 나은 경향을 보이는 seed를 사용하여 결과를 보고해도 됩니다.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/hopper.yaml --seed 48
python cas4160/scripts/run_hw4.py -cfg experiments/sac/hopper_doubleq.yaml --seed 48
python cas4160/scripts/run_hw4.py -cfg experiments/sac/hopper_clipq.yaml --seed 48
```

어느 것이 가장 잘 작동하나요? 여러분의 답변. 답변:

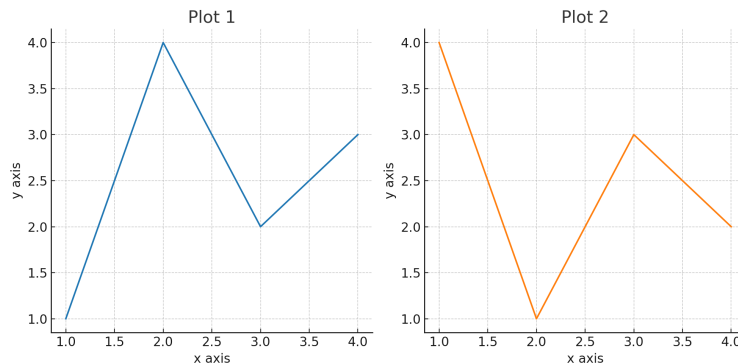


Figure 4: 이 플롯(`eval return`과 `Q-values`)을 여러분의 플롯으로 교체하고, 캡션을 추가하세요.

설명을 추가하세요.

- (선택 사항) single-Q, double-Q, clipped double-Q 중 가장 좋은 설정을 선택한 후, `humanoid.yaml`을 사용하여 Humanoid-v4 환경에서 실행하세요 (`config`를 편집하여 가장 좋은 옵션을 사용하세요). 이전 섹션의 기본 설정과 가장 좋은 설정을 함께 플롯으로 그리세요. x -축은 environment steps, y -축은 evaluation return입니다.

참고: 훈련을 끝까지(5M 스텝, 24시간 훈련) 실행하고 싶다면 제대로 걷는 humanoid를 얻을 수 있습니다! `-nvid 1`을 사용하여 비디오를 활성화하고 실행할 수 있습니다.

9 Submission

Please submit the code and the report in a single zip file, `hw4_[YourStudentID].zip`. Do not include videos and logs as the file size should be less than 20MB. The structure of the submission file should be:

```
hw4_[YourStudentID].zip
├── hw4_[YourStudentID].pdf
├── cas4160
│   ├── agents
│   │   └── sac_agent.py
│   └── ...
└── ...
```